

电子科技大学中山学院

University of Electronic Science and Technology of China , Zhongshan Institute

毕业设计 (论文)

外卖送餐员动态路径规划

教学单位: 计算机学院

专业名称: 软件工程

学　　号: 20180304020 79

学生姓名: 宋星润

指导教师: 赵梓权

指导单位: 计算机学院

完成时间: 2022 年 4 月 19 日

电子科技大学中山学院教务处制发

独创性声明

本人声明：所呈交的学位论文，是本人在指导老师的指导下独立进行研究工作所取得的研究成果。除文中特别加以标注、所列参考文献和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得电子科技大学中山学院或其它教育机构的学历、学位或课程、培训等非获奖类证书而使用过的成果材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。本人完全意识到本声明的法律后果由本人承担。

作者签名：_____　　日期：　　年　　月　　日

论文使用授权声明

本学位论文作者完全了解电子科技大学中山学院有关保留、使用学位论文的规定，本人同意学校有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权电子科技大学中山学院可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

(保密的学位论文在解密后应遵守此规定)

论文级别：本科论文

学科专业：

论文题目：

作者签名：_____　　导师签名：_____

日期：　　年　　月　　日

外卖送餐员动态路径规划

摘要

随着科技与社会的发展，人们的生活水平有了显著的提升，而外卖这一新兴行业也得到了极大的发展，而对于外卖员的送餐路径的规划将会影响这个行业的发展。动态路径规划是当前智能信息处理领域的前沿研究课题。外卖送餐员经常需要按照送餐地址送去餐品，所以需要更加规划的送餐路径使行驶的路程最短，就现有的动态路径规划问题提出更多解决方法的应用。完成了基于蒙特卡洛算法的外卖员的动态路径规划功能，能够在随机制作的地图中随着随机次数的上升找到越来越短路程的几率上升，并在多个地点中找到通往各个点的最短路径，从而减少外卖员的送餐时间成本，提高外卖员的效率和缩短顾客的等待时间。本动态路径规划功能使规划方法更加丰富。

关键词： 智能信息处理；蒙特卡洛算法；动态路径规划；

Dynamic path planning for food delivery workers

Abstract

With the development of technology and society, people's living standards have been significantly improved, and the emerging industry of food delivery has also been greatly developed. The planning of food delivery paths for takeaways will affect the development of this industry. Dynamic path planning is a frontier research topic in the field of intelligent information processing. Food delivery workers often need to deliver meals according to the delivery address, so a more planned delivery route is needed to make the travel distance the shortest, and the application of more solutions to the existing dynamic route planning problem is proposed. Completed the dynamic path planning function of the deliveryman based on the Monte Carlo algorithm, which can find the probability of finding shorter and shorter routes in the randomly made map as the random number increases, and find the way to each point in multiple locations The shortest path, thereby reducing the delivery time cost of the delivery staff, improving the efficiency of the delivery staff and shortening the waiting time of customers. This dynamic path planning function enriches the planning method.

Key Words: intelligent information processing; Monte Carlo method; dynamic path planning;

目录

第 1 章 绪论 ····· 1
1.1 课题背景 ····· 1
1.2 目的意义 ····· 2
1.2.1 研究目的 ····· 2
1.2.2 研究意义 ····· 2
1.2.3 研究范围和应解决的问题 ····· 2
1.3 论文主要工作 ····· 2
1.4 论文组织结构 ····· 2
第 2 章 相关技术和理论基础 ····· 3
2.1 技术与理论基础 ····· 3
2.2 matlab 语言 ····· 3
2.3 蒙特卡洛算法 ····· 3
2.4 数据结构 ····· 4
第 3 章 基于蒙特卡洛算法的路径规划基础 ····· 5
3.1 图 ····· 5
3.2 递归蒙特卡洛算法的实现 ····· 6
3.3 循环蒙特卡洛算法的实现 ····· 7
3.4 随机地图的制作和字母地址与数字地址的转换 ····· 9
3.5 实验环境 ····· 10
3.6 本章小结 ····· 10
第 4 章 基于蒙特卡洛算法的动态路径规划和算法优化 ····· 11
4.1 地图预载优化和预分配内存优化 ····· 11
4.2 搜索范围优化 ····· 12
4.3 实现动态路径规划 ····· 16
4.4 本章小结 ····· 17
第 5 章 测试与分析 ····· 18
5.1 地图预载与内存预分配优化 ····· 18
5.2 搜索范围优化 ····· 19
5.3 搜索范围优化 ····· 20
第 6 章 总结和展望 ····· 22
6.1 本文总结 ····· 22
6.2 未来展望 ····· 22
参考文献 ····· 23

致谢 24

图目录

图 3-1 带权有向图和它的邻接矩阵 ····· 5
图 3-2 递归蒙特卡洛算法流程图 ····· 6
图 3-3 循环蒙特卡洛算法流程图 ····· 8
图 4-1 地图前置路径规划流程图 ····· 11
图 4-2 搜索范围优化路径规划流程图 ····· 13
图 4-3 动态路径规划流程图 ····· 16
图 5-1 26 点测试地图矩阵 ····· 18

表目录

表 5-1 地图预载与内存预分配优化测试表 ····· 19
表 5-2 搜索范围优化测试表 ····· 20
表 5-3 搜索范围优化动态路径测试表 ····· 21

第 1 章 绪论

1.1 课题背景

近年来, 国内经济的迅速发展带来了交通需求的日益增长, 交通拥堵已经成为大中城市交通的普遍现象, 这影响了人们出行的安全与效率，也使一些需要交通的行业面临许多困难。现在有许多路径规划的算法可以解决这些问题，本课题是提出更多的对交通问题的解决办法。外卖员动态路径规划是在短时间内给出一条最优或近似最优的出行路径, 这对路径规划算法的实时性和准确性提出了很高的要求。

动态路径规划是我国智能交通系统和地理信息系统当中所关注的重点问题。而全球上从最早的最短路径算法 Dijkstra 算法, 到后来的 Floyd 算法，A* 算法，D* 算法，LPA* 算法和 D*lite 算法都在寻求更加便利的算法来解决动态路径规划这个问题。

别人已经研究的工作包括：

1. Dijkstra 算法是由 E.W.Dijkstra 于 1959 年提出，又叫迪杰斯特拉算法。该算法采用了一种贪心模式，其解决的是有向图中单个节点到另一节点的最短路径问题，其主要特点是每次迭代时选择的下一个节点是当前节点最近的子节点，也就是说每一次迭代行进的路程是最短的。而为了保证最终搜寻到的路径最短，在每一次迭代过程中，都要对起始节点到所有遍历到的点之间的最短路径进行更新。

2. A* 算法是启发式搜索算法，启发式搜索即在搜索过程中建立启发式搜索规则，以此来衡量实时搜索位置和目标位置的距离关系，使搜索方向优先朝向目标点所处位置的方向，最终达到提高搜索效率的效果。

3. 基于 A* 算法，Anthony Stentz 在 1994 年提出了 Dynamic A* 算法，也就是 D* 算法。D* 算法是一种反向增量式搜索算法，反向即算法从目标点开始向起点逐步搜索；增量式搜索，即算法在搜索过程中会计算每一个节点的距离度量信息 H(x)，在动态环境中若出现障碍物无法继续沿预先路径搜索，算法会根据原先已经得到的每个点的距离度量信息在当前状态点进行路径再规划，无需从目标点进行重新规划。

4. 蒙特卡洛算法, 也称统计模拟方法，是二十世纪四十年代中期由于科学技术的发展和电子计算机的发明，而被提出的一种以概率统计理论为指导的一类非常重要的数值计算方法。

我完成的工作：

1. 基于蒙特卡洛算法完成最短路径规划

(a) 基于蒙特卡洛算法完成最短路径规划的基本代码。

(b) 完成随机地图生成代码。

2. 优化路径规划代码

(a) 从递归优化为循环中优化代码的空间复杂度。

(b) 从预分配内存，循环层数和次数的减少和前置代码的运行来减少代码的时间复杂度。

3. 最短路径规划代码改进为动态路径规划代码

(a) 目的地变为多个地点。再进行路径规划算法。

(b) 增大地图的大小。

(c) 省略不必要的循环再次提高代码的运行速度。

1.2 目的意义

1.2.1 研究目的

用蒙特卡洛算法实现动态路径规划的功能，从读取的地图中找到最优解或近似最优解。

1.2.2 研究意义

此动态路径规划功能为路径规划问题提出了一个新的解决方法。此功能为送餐员送餐路径的规划更加快速。在一定次数的随即下路线近似最优的路径规划功能。

1.2.3 研究范围和应解决的问题

主要研究动态的路径规划，从中找到最快速的方法。要同时保证速度与最优解。

1.3 论文主要工作

介绍本研究课题的来源及主要研究内容。

论文的背景见1.1，论文的目的意义见1.2。

1.4 论文组织结构

第 1 章介绍了考研教室预约系统的课题背景，目的意义，组员分工，全文的组织结构。

第 2 章介绍了外卖送餐员动态路径规划功能的相关技术。包括 matlab 语言，蒙特卡洛算法，数据结构。

第 3 章介绍了对基于蒙特卡洛算法对路径规划的基础，路径规划的基本实现，辅助功能如随机地图制作和地址和数字矩阵的转换等功能的叙述。

第 4 章介绍了基于蒙特卡洛算法的路径规划算法的各种优化方案，如地图预载，预分配内存，搜索范围优化等和外卖送餐员的动态路径规划方案。

第 5 章讲述了本文算法的测试和各个优化对于算法的提升与分析。

第 6 章总结和展望，总结了算法的开发工作，分析了该功能目前存在的问题及系统需要进一步完善的地方。

第 2 章 相关技术和理论基础

2.1 技术与理论基础

外卖员动态路径规划用到了 matlab 语言，蒙特卡洛算法，数据结构。

2.2 matlab 语言

MATLAB 语言总是与数学计算联系在一起的，于是当然其产生也与数学计算有着紧密的联系。1980 年，美国新墨西哥州大学计算机系主任 Cleve Moler 在给学生讲授线性代数课程时，发现学生在高级语言编程上花费了很多时间，于是决定编写供学生使用的 Fortran 子程序库接口程序，他将其命名为 MATLAB（即 Matrix Laboratory 的前三个字母的组合，意为”矩阵实验室“）。

2.3 蒙特卡洛算法

蒙特卡罗方法（Monte Carlo method），也称统计模拟方法，是二十世纪四十年代中期由于科学技术的发展和电子计算机的发明，而被提出的一种以概率统计理论为指导的一类非常重要的数值计算方法。是指使用随机数（或伪随机数）来解决很多计算问题的方法。与它对应的是确定性算法。蒙特卡罗方法在金融工程学，宏观经济学，计算物理学（如粒子输运计算、量子热力学计算、空气动力学计算）等领域应用广泛。另外，拟蒙特卡洛算法在近几年也获得迅速发展。这种方法是用确定性的超均匀分布代替蒙特卡洛算法中的随机数序列，对于某些特定问题计算速度比普通的蒙特卡洛算法高几百倍。

由于产生随机数的随机性，当我们用 N 个随机点以蒙特卡罗方法来求解具体的问题时，其计算得到近似解的误差值有大有小，但是肯定有一个确定的平均值，即一些误差大于此值，而其余误差小于此值。鉴于此，显然肯定存在这样的 N 个点，使得误差的绝对值不大于平均值。如果我们能够构造这样的点集，就可以对原有的方法进行较大的改进。拟蒙特卡罗方法就是至于此而提出的，它致力于构造其误差比平均误差显著要好的那种点集，而其求解形式与蒙特卡罗方法一致，只不过所用的随机数不一样。用蒙特卡罗方法求解问题时，影响结果好坏的主要是随机数序列的均匀性。而拟蒙特卡罗方法中的具有低偏差的一致分布点集较伪随机数序列更为均匀，而且用拟蒙特卡罗方法求解得到的是真正的误差，避免了蒙特卡罗方法得到概率误差的缺陷。

由此可见用拟蒙特卡罗方法求解问题的关键是如何找到一个均匀散布的点集。目前常用的点集有 GLP 点集 (好格子点集，good lattice point set)、GP 点集 (好点集，good point set)、Halton 点集及其变体、Hammersley 点集等。

蒙特卡洛方法的理论基础是大数定律。大数定律是描述相当多次数重复试验的结果的定律，根据这个定律知道样本数量越多，其平均就越趋近于真实值。

2.4 数据结构

数据结构 (data structure) 是带有结构特性的数据元素的集合，它研究的是数据的逻辑结构和数据的物理结构以及它们之间的相互关系，并对这种结构定义相适应的运算，设计出相应的算法，并确保经过这些运算以后所得到的新结构仍保持原来的结构类型。简而言之，数据结构是相互之间存在一种或多种特定关系的数据元素的集合，即带“结构”的数据元素的集合。“结构”就是指数据元素之间存在的关系，分为逻辑结构和存储结构。数据的逻辑结构和物理结构是数据结构的两个密切相关的方面，同一逻辑结构可以对应不同的存储结构。算法的设计取决于数据的逻辑结构，而算法的实现依赖于指定的存储结构。数据结构的研究内容是构造复杂软件系统的基础，它的核心技术是分解与抽象。通过分解可以划分出数据的 3 个层次；再通过抽象，舍弃数据元素的具体内容，就得到逻辑结构。类似地，通过分解将处理要求划分成各种功能，再通过抽象舍弃实现细节，就得到运算的定义。上述两个方面的结合可以将问题变换为数据结构。这是一个从具体（即具体问题）到抽象（即数据结构）的过程。然后，通过增加对实现细节的考虑进一步得到存储结构和实现运算，从而完成设计任务。这是一个从抽象（即数据结构）到具体（即具体实现）的过程。

第 3 章　基于蒙特卡洛算法的路径规划基础

3.1　图

图 (Graph) 是由若干给定的顶点及连接两顶点的边所构成的图形，这种图形通常用来描述某些事物之间的某种特定关系。顶点用于代表事物，连接两顶点的边则用于表示两个事物间具有这种关系。

无向图：顶点之间相连的线我们称为边，每条边都是无方向的。

有向图：顶点之间相连的线我们称为弧，每条弧都是有方向的。

网：边/弧带权值的图。

权与网：图中边或弧所具有的相关数称为权。表明从一个顶点到另一个顶点的距离或耗费。带权的图称为网。

在本文实验中使用带权有向图作为动态路径规划的地图，并用带权有向图的邻接矩阵来完成地图和进行运算。如图3-1所示。

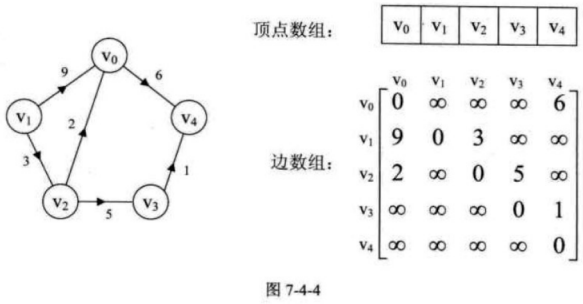


图 3-1 带权有向图和它的邻接矩阵

在程序中加入语句来找出地图中每个地点的可行路径数，以便蒙特卡洛算法使用的随机数的产生。

如下面代码所示

```
for  i=1:x
    for  j=1:x                    %循环遍历每一个地点
        if  G(i,j)~=0                    %当某一地点的一路径是通路时
            point(i)=point(i)+1;                    %那么那一地点的可行数+1
        end
    end
end
```

3.2　递归蒙特卡洛算法的实现

蒙特卡洛算法实现路径寻找相当于每一次在现在的地点随机选择一个能够走的道路行进，一直重复这个过程直到到达目的地，这个路径就是一条从始发地到目的地的通路。

所以蒙特卡洛算法实现路径规划是使用随机数尽可能找到最短路径，当随机次数越多找到的通路数就会越多，再把找到的路径互相比较，就越有可能找到最短的路径。

1.while 循环为要进行几次蒙特卡洛算法次数，并把每一次递归算法出来的路径和距离与之前递归算法得到的最小值做对比，并判断是否替换。

2. 读取当前所在地的地址，并根据当前所在地和目的地的位置做判断，是否要做递归算法，如果一开始始发地与目的地就相等的话，就不会进行下面的递归算法。

3. 当开始递归算法时，使用随机数选择将要行进的通路，并把下一个地点的位置和终点的位置作为参数进行下一次递归。

4. 当当前所在地与目的地相等时，递归就会一步一步的把距离与路径累加并返还。如图3-2所示。

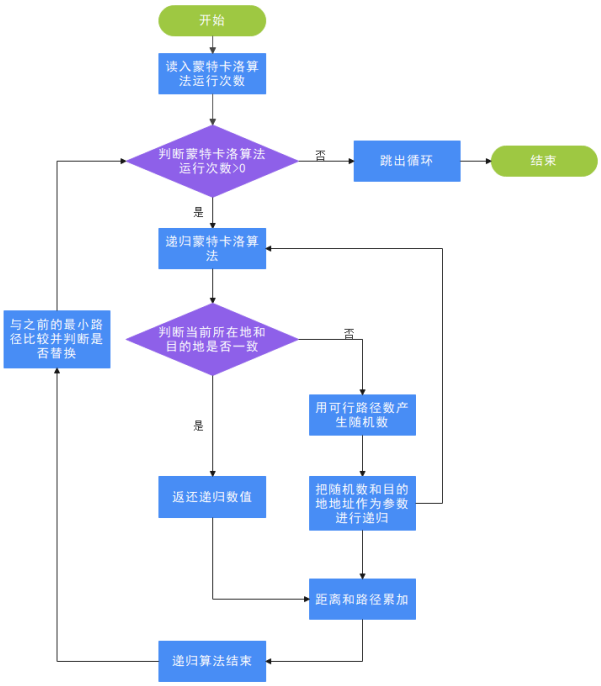


图 3-2 递归蒙特卡洛算法流程图

如下面代码所示。

```
while  replication_number>=0                    %开始蒙特卡洛算法次数循环
    [once_distance ,path_once]=recursion(now_number1,destination_number);
        %调用递归算法
    if  min_distance>=once_distance%每一次的循环得到的距离会和之前得到的最小距离作
        比较并替换
        min_distance=once_distance;
```

```
            path_szmin1=path_once;
        end
        replication_number=replication_number-1;                    %循环次数减一
    end
    function [A,B]= recursion(now_number,destination_number)        %递归蒙特卡洛算法
        if now_number==destination_number
            A=0;
            B=[ ];
        else
            real_number=0;                  %定义随着选择数字判断非零数字的地址
            mtkl_number=randi(point(now_number));  %随机一个根据地点的可行路径数的非
                零整数
            for search=1:x                  %开始循环遍历当前所在地址的每一个路口
                if G(now_number,search)~=0%如果某一路口是非零通路那么通路地址就会
                    前进一位
                    real_number=real_number+1;
                end
                if real_number==mtkl_number         %当通路地址和随机地址相等时
                    [A,B]=recursion(search,destination_number);          %递归循环
                    A=G(now_number,search)+A;           %距离增加
                    B=[B search];               %路径累加
                    break;
                end
            end
        end
    end
```

3.3　循环蒙特卡洛算法的实现

由于递归算法每一次的递归都将会重新划分内存赋值变量，所以使用循环语句替代它降低它的空间复杂度。

1. 建立三重循环，第一重循环为蒙特卡洛随机次数，并把每一次循环出来的路径和距离与之前循环得到的最小值做对比，并判断是否替换。

2. 第二重循环为找到一条从始发地到目的地的路径并把距离累加起来，再跳出二重循环。

3. 第三重循环为循环遍历当前所在地的每一个每一个路口，查找和随机数相同的路口并把到达的路口地址赋给现在的地址。

如图3-3所示。

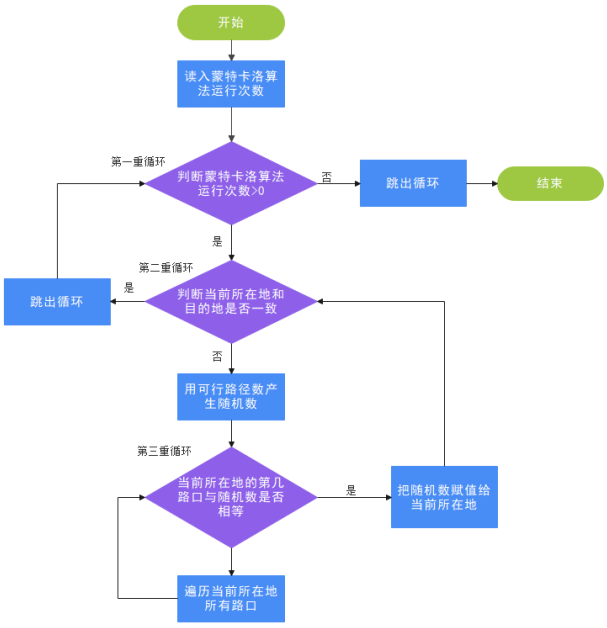


图 3-3 循环蒙特卡洛算法流程图

如下面代码所示。

```
while  replication_number>=0                    %开始蒙特卡洛算法次数循环
    once_distance=0;                    %每一遍循环的距离清零
    now_number=now_number1;             %每一遍循环把现在地址重新设为始发地数字地址
    path=[now_number];                  %定义每一遍循环时经过地点的数组
    if  destination_number==now_number1                 %当目的地和始发地相等时跳出循环
        min_distance=0;
        path_szmin=path;
        break;
    end
    while  ~0                   %查找能到目的地路径和距离的循环
        real_number=0;                  %定义随着选择数字判断非零数字的地址
        mtkl_number=randi(point(now_number));       %随机一个根据地点的可行路径数的非
            零整数
        for  search=1:x                 %开始循环遍历当前所在地址的每一个路口
            if  G(now_number,search)~=0     %如果某一路口是非零通路那么通路地址就会前
                进一位
                real_number=real_number+1;
            end
            if  real_number==mtkl_number                %当通路地址和随机地址相等时
                once_distance=G(now_number,search)+once_distance;               %距离会累
                    加
                now_number=search;              %到达的地点数字会赋给现在的地点数字
                path=[path  search ];               %经过地点的数字会累加
                break;
            end
```

```
            end
            if now_number==destination_number          %当现在的地址到达目的地时跳出循环
                break;
            end
    end
        if once_distance<min_distance%每一次的循环得到的距离会和之前得到的最小距离作比
            较并替换
            min_distance=once_distance;
            path_szmin=path;                    %经过路径进行替换
        end
        replication_number=replication_number-1;                %循环次数减一
    end
```

3.4　随机地图的制作和字母地址与数字地址的转换

每个地方的地图都应该是不一样的，所以完成了一个随机生成地图的代码。

每个地点除了与自身的距离为 0，别的通路均为 0 到 20 的距离。

如下面代码所示。

```
function G=automatic_map_generation(number)                %自动生成地图
    map=zeros(number);
    rng('shuffle');
    for x=1:number
        for y=1:number
            suiji_juli =randi ([0,20],1) ;                  %随机从0到20生成距离长度
            if x==y                 %如果路径是通向自身则路径距离为0
                map(x,y)=0;
                continue;
            end
            map(x,y)= suiji_juli ;                  %把随机值赋到矩阵上
        end
    end
    G=map;                  %返回地图矩阵
end
```

矩阵地图的显示应该是数字的，但正常的地址应该为字符或字符串。

所以当始发地与目的地为字符或字符串时，将会把字母地址转变为数字地址进行算法运算，当运算完成时，便会将数字地址转变为字母地址。

如下面部分代码所示。

```
[~,o]=size(path_szmin);                 %计算经过路径数字的数量
for p=1:o
    path_min(p)=sz_transform(path_szmin(p));                %把经过路径的数字地址转化成字
        母地址
```

```
end
    function  site  =sz_transform( szloc )                    %数字地址转化成字母地址
        switch  szloc
            case  1
                site ='A';
            case  2
                site ='B';
            case  3
                site ='C';
            case  4
        end
    end
```

3.5　实验环境

在本文实验中
硬件为 CPU 为 Intel(R) Core(TM) i7-9700K CPU @ 3.60GHz
GPU 为 NVIDIA GeForce RTX 2070 显存为 16GB
软件平台包括：Windows 10 专业版 64 位操作系统，MATLAB R2016a

3.6　本章小结

本章主要介绍了图的使用来表示地图和地图中地点的两种表示和随机地图的生成。详述了递归和循环在蒙特卡洛算法中的运用。介绍了本文的实验环境。为后续章节中的蒙特卡洛算法优化提供了基础。

第 4 章　基于蒙特卡洛算法的动态路径规划和算法优化

4.1　地图预载优化和预分配内存优化

在地图生成后，把矩阵地图在前置代码中运行，返回地图的每个地点的可行路径数，和表示是第几条路径的可行路径矩阵。这样只用第一次更换地图时运行一遍前置代码，去掉了了每次蒙特卡洛算法时开始的地图遍历，也使三重循环变成了二重循环。减少了时间复杂度。

如图4-1所示。

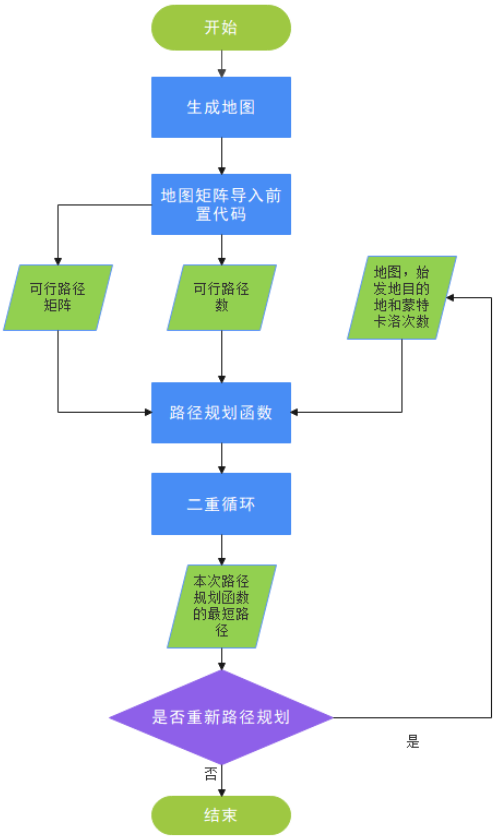


图 4-1 地图前置路径规划流程图

如下面代码所示。

```
function  [W,Q]= precondition(G)                    %读出地图上每个地点可行路准确位置
[x,~]=size(G);                                      %地图的尺寸
point=zeros([1  x]);                                %每个地点的可行路径数数组
feasible_way=zeros(x);                              %记录每个地点的可行路径是第几条路的矩阵
for  i=1:x                                          %循环遍历每一个点
    z=1;
```

```
    for  j=1:x
        if  G(i,j)~=0
            point(i)=point(i)+1;          %当某个地点的某条路径为通路时，可行路径数+1
            feasible_way(i,z)=j;              %矩阵记录这个可行路径是第几条路
            z=z+1;
        end
    end
end
W=feasible_way;                  %返回可行路径矩阵
Q=point;                  %返回可行路径数
end
```

在 matlab 中，定义的数组不能为空值，所以每次添加新的行走路线都是在数组的最后分配内存添加数据，在循环中这样会随着循环次数不停的分配内存，极大的减慢了代码的运行速度。所以定义了一个长度 26 全为 0 的数组用于存入行走的路径。并在每次从始发地开始时，把存入的下标定在 2。在进行累加。并在最后把所求的最短路径从数组中截取出来。再转换成字符串地址。

4.2　搜索范围优化

在现实生活中，地图上的地点是非常多的，所以当地点变多时，算法的时间复杂度就会极大地提高。在本文的实验中也由 26 个地点提高到 50 个点左右，但运行时间却极大的上升了，最短路径准确率也低了很多。

所以缩小了首次搜索的范围，先把地图导入到前置函数中，除了返回可行路径数和可行路径矩阵。还把比地图中最长路径一半的路径单独制成可行路径矩阵和可行路径数。然后共同导入到路径规划函数中。在路径规划函数中，首先先从缩小后的地图中蒙特卡洛算法搜索最短路径，当最短路程没有找到或超过 20 时，就会再从完整的地图中去寻找最短路径。而在寻路过程中，如果蒙特卡洛随机了 15 次都没有到达目的地就会直接跳出这次循环，开始下一次的寻路。这样在总体上极大的减少了算法的时间复杂度。

如图4-2所示。

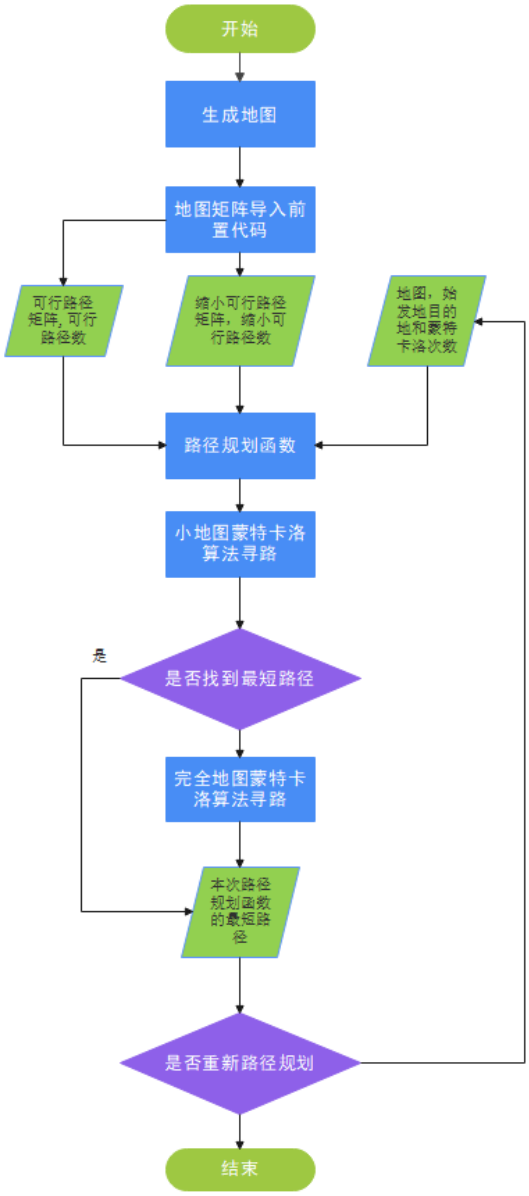


图 4-2 搜索范围优化路径规划流程图

如下面代码所示。

```
function  [W,Q,min_W,min_Q]= precondition_updated(G) %读出地图上每个地点可行路准确位置
    [x,~]=size(G);                    %地图的尺寸
    point=zeros([1  x]);                  %每个地点的可行路径数数组
    min_point=zeros([1  x]);
    feasible_way=zeros(x);                  %记录每个地点的可行路径是第几条路的矩阵
    min_feasible_way=zeros(x);
    for  i=1:x                  %循环遍历每一个点
        z=1;
        a=1;
        for  j=1:x
            if  G(i,j)~=0
```

```
                point(i)=point(i)+1;     %当某个地点的某条路径为通路时，可行路径数+1
                feasible_way(i,z)=j;                %矩阵记录这个可行路径是第几条路
                if G(i,j)<=10                %当路线距离小于10存入到短路径地图里
                    min_feasible_way(i,a)=j;
                    min_point(i)=min_point(i)+1;
                    a=a+1;
                end
                z=z+1;
            end
        end
    end
    W=feasible_way;                %返回可行路径矩阵
    Q=point;                %返回可行路径数
    min_W=min_feasible_way;
    min_Q=min_point;
end
```

```
while replication_number>=0                %开始小范围蒙特卡洛算法次数循环
    replication_number=replication_number-1;                %循环次数减一
    once_distance=0;                %每一遍循环的距离清零
    rep_num=0;
    now_number=now_number1;                %每一遍循环把现在地址重新设为始发地数字地址
    path(1)=now_number;                %定义每一遍循环时经过地点的数组
    path_num=1;
    if destination_number==now_number1                %当目的地和始发地相等时跳出循环
        min_distance=0;
        path_szmin=path;
        break;
    end
    while ~0                %查找能到目的地路径和距离的循环
        rep_num=rep_num+1;
        mtkl_number=randi(min_point(now_number));%随机一个根据地点的可行路径数的非零整
            数
        once_distance=G(now_number,min_feasible_way(now_number,mtkl_number))+
            once_distance;                %距离累加
        path_num=path_num+1;
        path(path_num)=min_feasible_way(now_number,mtkl_number);                %路径累加
        now_number=min_feasible_way(now_number,mtkl_number);%现在数字地址被随机数选中
            的地址替代
        if now_number==destination_number     %当现在的地址到达目的地时跳出循环
            break;
        end
        if rep_num>=15
            break;
```

```
        end
    end
    if rep_num>=15
        continue;
    end
    if once_distance<min_distance%每一次的循环得到的距离会和之前得到的最小距离作比较并
        替换
        min_distance=once_distance;
        path_szmin=path;                    %经过路径进行替换
    end
end
if min_distance>=20
    while replication_number>=0                %开始全图蒙特卡洛算法次数循环
        replication_number=replication_number-1;                %循环次数减一
        once_distance=0;                    %每一遍循环的距离清零
        rep_num=0;
        now_number=now_number1;%每一遍循环把现在地址重新设为始发地数字地址
        path(1)=now_number;                 %定义每一遍循环时经过地点的数组
        path_num=1;
        if destination_number==now_number1          %当目的地和始发地相等时跳出循环
            min_distance=0;
            path_szmin=path;
            break;
        end
        while ~0                    %查找能到目的地路径和距离的循环
            rep_num=rep_num+1;
            mtkl_number=randi(point(now_number));%随机一个根据地点的可行路径数的非零整
                数
            once_distance=G(now_number,feasible_way(now_number,mtkl_number))+once_distance
                ;               %距离累加
            path_num=path_num+1;
            path(path_num)=feasible_way(now_number,mtkl_number);            %路径累加
            now_number=feasible_way(now_number,mtkl_number);%现在数字地址被随机数选中
                的地址替代
            if now_number==destination_number   %当现在的地址到达目的地时跳出循环
                break;
            end
            if rep_num>=15
                break;
            end
        end
        if rep_num>=15
            continue;
        end
```

```
        if  once_distance<min_distance     %每一次的循环得到的距离会和之前得到的最小距离
            作比较并替换
            min_distance=once_distance;
            path_szmin=path;                    %经过路径进行替换
        end
    end
end
```

4.3　实现动态路径规划

外卖送餐员每一次送餐大概率不是一次一单的，所以当有多个地点需要送达时，需要规划出更加合理的路线来让送餐员最快时间内经过两个地点。所以我们需要将两个地点进行比较先去哪里路径最短，在返回最短的路径为最短路径。如图4-3所示。

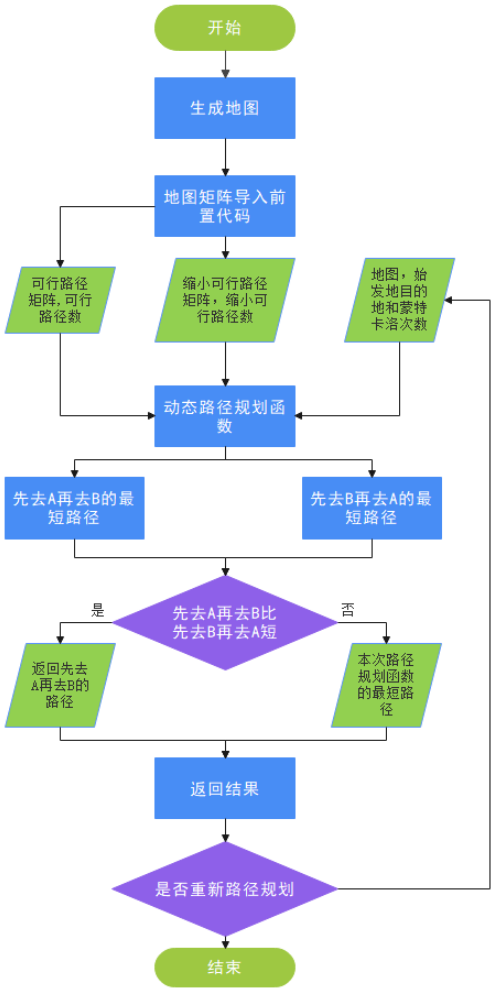


图 4-3 动态路径规划流程图

如下面代码所示。

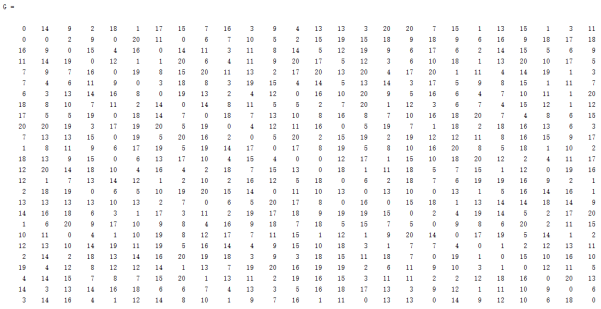
```
[ first_mindistance_1 , first_minpath_1 ]=mtkl_updated_pro_enhance(map,location, destination_A ,
    running_times, feasible_way , point ,min_feasible_way,min_point);%计算始发地地址到第一个
    地点的最短路径
[ first_mindistance_2 , first_minpath_2 ]=mtkl_updated_pro_enhance(map,destination_A,
    destination_B ,running_times, feasible_way , point ,min_feasible_way,min_point);%计算第一个
    地点到第二个地点的最短路径
 first_mindistance = first_mindistance_1 + first_mindistance_2 ;
 first_minpath =[ first_minpath_1 (1:end-1)  first_minpath_2 ];
[second_mindistance_1,second_minpath_1]=mtkl_updated_pro_enhance(map,location,destination_B ,
    running_times, feasible_way , point ,min_feasible_way,min_point);%计算始发地到第二个地点
    的最短路径
[second_mindistance_2,second_minpath_2]=mtkl_updated_pro_enhance(map,destination_B,
    destination_A,running_times, feasible_way , point ,min_feasible_way,min_point);%计算第二个
    地点到第一个地点的最短路径
second_mindistance=second_mindistance_1+second_mindistance_2;
second_minpath=[second_minpath_1(1:end-1) second_minpath_2];
 if  first_mindistance >=second_mindistance%对比先去第一个地点和先去第二个地点的距离
    Y= first_mindistance ;%并返回最短距离
     first_minpath
 else
    Y=second_mindistance;
    second_minpath
end
```

4.4　本章小结

本章主要介绍了使用前置代码将地图预载，预分配内存加快运行速度。详述了搜索范围优化算法的细节，介绍了动态路径规划的方法。为后续章节中的测试和分析提供了基础。

第 5 章 测试与分析

本文实验是由随机生成地图作为测试地图
如图5-1所示。为 26 个点的地图



0	14	9	2	18	1	17	15	7	16	3	9	4	13	13	3	20	20	7	15	1	13	15	1	3	11
0	0	2	9	0	20	11	0	6	7	10	8	2	15	19	16	18	9	18	9	6	16	9	18	17	18
16	9	0	18	4	16	0	14	11	3	11	8	14	5	12	19	9	6	17	6	2	14	15	5	6	9
11	14	19	0	12	1	1	20	6	4	11	9	20	17	5	12	3	6	10	18	1	13	20	10	17	5
7	9	7	16	0	19	8	15	20	11	13	2	17	20	13	20	4	17	20	1	11	4	14	19	1	3
7	4	6	11	9	0	3	18	8	3	19	15	4	14	5	13	14	3	17	5	9	8	15	1	11	7
6	3	13	14	16	8	0	19	13	2	4	12	0	16	10	20	9	5	16	6	4	7	10	11	1	20
18	8	10	7	11	2	14	0	14	8	11	5	5	2	7	20	1	12	3	6	7	4	15	12	1	12
17	5	5	19	0	18	14	7	0	18	7	13	10	8	16	8	7	10	16	18	20	7	4	8	6	15
20	20	19	3	17	19	20	5	19	0	4	12	11	16	0	5	19	7	1	18	2	18	16	13	6	3
7	13	13	15	0	19	5	20	16	2	0	5	20	2	15	19	2	19	12	12	11	8	16	15	9	17
1	8	11	9	6	17	19	5	19	14	17	0	17	6	19	5	8	10	16	20	8	5	18	1	10	2
18	13	9	15	0	6	13	17	10	4	15	4	0	0	12	17	1	15	10	18	20	12	2	4	11	17
12	20	14	18	10	4	16	4	2	18	7	15	13	0	18	1	11	18	5	7	15	1	12	0	19	16
12	1	7	13	14	12	1	2	10	2	16	12	5	18	0	6	2	18	7	6	19	19	16	9	2	1
2	18	19	0	6	5	10	19	20	15	14	0	11	10	13	0	13	10	0	13	1	5	16	14	16	1
13	13	13	13	10	13	2	7	0	6	5	20	17	8	0	16	0	15	18	1	13	14	14	18	14	9
14	16	18	6	3	1	17	3	11	2	19	17	18	9	19	19	15	0	2	4	19	14	5	2	17	20
1	6	20	9	17	10	9	8	4	16	9	18	7	18	5	15	7	5	0	9	8	6	20	2	11	15
10	11	0	4	1	10	19	8	12	17	7	11	15	1	12	1	9	20	14	0	17	19	5	14	1	2
12	13	10	14	19	11	19	5	16	14	4	9	15	10	18	3	1	7	7	4	0	1	2	12	13	11
2	14	2	18	13	14	16	20	19	18	3	9	3	18	15	11	18	7	0	19	1	0	15	10	16	10
19	4	12	8	12	12	14	1	13	7	19	20	16	19	19	2	6	11	9	10	3	1	0	12	11	5
4	14	15	7	8	7	15	20	1	13	11	2	19	16	15	3	11	11	2	2	12	18	16	0	20	13
14	3	13	14	16	18	6	6	7	4	13	3	5	16	18	17	13	3	9	12	1	11	10	9	0	6
3	14	16	4	1	12	14	8	10	1	9	7	16	1	11	0	13	13	0	14	9	12	10	6	18	0

图 5-1 26 点测试地图矩阵

一共做了 3 次图表对比分析，分别为
1. 是否地图预载，内存预分配优化。
2. 是否动态搜索范围优化。
3. 动态路径规划下时，是否动态搜索范围优化。
第一次图表对比所用 26 点地图。
所用地址为大写英文字母 A 到 Z。
第二次和第三次图表对比所用 50 点地图。
所用地址为元素周期表上的 1 号元素 H 到 50 号元素 Sn。
由于 50 点地图矩阵过大，再本文中不方便展示。
每个表格一共 7 列，第 2 到 4 列和第 5 到 7 列表示的含义是相同的。
第一列为每次图表所测试的次数，总共为 20 次。
第二列为每次的规划路径结果。
第三列为每次的所得路径的路程，单位为百米。
第四列为每次的运行时间，单位为秒。

5.1 地图预载与内存预分配优化

如表5-1所示
mtkl 函数为未优化的蒙特卡洛路径规划算法。
pro 函数为经过地图预载和内存预分配优化后的蒙特卡洛路径规划算法。
由表可知

在未优化的蒙特卡洛算法中，20000 次的蒙特卡洛随机寻路下，最短距离稳定在 4 百米。运行时间在 0.72 秒左右。未出现规划路径未找到最短路径这种状况。表明在 26 点地图上，20000 次未优化的路径规划数据稳定。

在经过地图预载和内存预分配优化后，在最短距离稳定的情况下，运行时间减少到了 0.24 秒左右。大约减少了 0.48 秒，运行速率提升了 2 倍。

表 5-1 地图预载与内存预分配优化测试表

	H 到 Z/20000 次		mtkl 函数	H 到 Z/20000 次		pro 函数
1	HNPZ	4	0.718	HNPZ	4	0.263
2	HNPZ	4	0.709	HNPZ	4	0.250
3	HNPZ	4	0.746	HNPZ	4	0.255
4	HNPZ	4	0.722	HNPZ	4	0.254
5	HNPZ	4	0.693	HNPZ	4	0.243
6	HNPZ	4	0.739	HNPZ	4	0.261
7	HNPZ	4	0.727	HNPZ	4	0.256
8	HNPZ	4	0.719	HNPZ	4	0.252
9	HNPZ	4	0.867	HNPZ	4	0.252
10	HNPZ	4	0.722	HNPZ	4	0.257
11	HNPZ	4	0.705	HNPZ	4	0.259
12	HNPZ	4	0.705	HNPZ	4	0.245
13	HNPZ	4	0.721	HNPZ	4	0.245
14	HNPZ	4	0.726	HNPZ	4	0.252
15	HNPZ	4	0.708	HNPZ	4	0.247
16	HNPZ	4	0.720	HNPZ	4	0.242
17	HNPZ	4	0.731	HNPZ	4	0.258
18	HNPZ	4	0.707	HNPZ	4	0.251
19	HNPZ	4	0.723	HNPZ	4	0.226
20	HNPZ	4	0.722	HNPZ	4	0.245

5.2 搜索范围优化

如表5-2所示

pro 函数为经过前面优化但未经过搜索范围优化的函数。

proenhance 为经过搜索范围优化的函数。

此图为 50 个点的测试地图。

由表可见，没有优化前，在前 10 次测试时，由于数据极不稳定，路程和路径变动过大，所以在 10 次之后将蒙特卡洛随机次数提高到了 100000 次。但运行时间也大大增加。

路程由不稳定的状态下变为大概稳定在 4 百米上。时间从 0.49 左右上升到 2.4 左右。虽然时间成本增加了，但是稳定性也大大提高了。并且路程近似最短距离。

在优化后，前十次测试路径与路程都比较稳定。运行时间从未优化的 0.49 左右减少到了 0.14 左右。在把蒙特卡洛随机次数提高到了 100000 次之后，在运行时间没有增加太多的情况下，再次提高稳定性。

表 5-2 搜索范围优化测试表

	He 到 Fe/20000 次		pro 函数	He 到 Fe/20000 次		proenhance 函数
1	HeBrSrFe	5	0.502	HeAsNaFe	4	0.127
2	HeAsNaFe	4	0.500	HeAsNaRhFe	4	0.145
3	HeInFe	8	0.503	HeAsNaFe	4	0.144
4	HeBrSrFe	5	0.497	HeAsNaFe	4	0.144
5	HeBrSrFe	5	0.497	HeAsNaFe	4	0.140
6	HeBrSrFe	5	0.498	HePAsNaFe	5	0.140
7	HeAsNaFe	4	0.497	HeAsNaFe	4	0.142
8	HeInFe	8	0.502	HeAsNaFe	4	0.141
9	HeBrSrFe	5	0.428	HeAsNaFe	4	0.145
10	HeBrSrFe	5	0.399	HeAsNaFe	4	0.144
11	HeAsNaFe	4	2.442	HeAsNaFe	4	0.651
12	HeAsAlFe	5	2.413	HePdRhFe	5	0.643
13	HeAsNaFe	4	2.407	HeAsNaFe	4	0.650
14	HeAsNaFe	4	2.415	HeAsNaFe	4	0.657
15	HeAsNaFe	4	2.435	HeAsNaFe	4	0.661
16	HeAsNaFe	4	2.399	HeAsNaRhFe	4	0.641
17	HeAsNaFe	4	2.427	HeAsNaFe	4	0.656
18	HeAsNaFe	4	2.430	HeAsNaFe	4	0.647
19	HeAsNaFe	4	2.442	HeAsNaFe	4	0.653
20	HeAsAlFe	5	2.415	HeAsNaFe	4	0.647

5.3 搜索范围优化

如表5-3所示。

pro 函数为经过前面优化但未经过搜索范围优化的动态路径规划函数。

proenhance 为经过搜索范围优化的动态路径规划函数。

由表可见，在未优化之前，前十次测试中，所得最短路程在 9 百米和 7 百米中来回跳动，路径也一直不停的变换，数据不够稳定，运行时间在 1.9 秒上下浮动，所以为了提高稳定性，把蒙特卡洛随机次数提高到 100000 次，在提升次数之后，路程稳定在了 7

百米，但由于循环次数也大大增加，时间由 1.9 秒上升到了 9.4 秒左右。虽然稳定性得到了提高，但运行速度过慢。

在经过搜索范围优化之后，无论在蒙特卡洛随机次数提高前，还是提高后，所得数据都比较稳定。运行时间也比优化前大大减少。

表 5-3 搜索范围优化动态路径测试表

	He 到 Ge 到 Cd		pro 函数			proenhance 函数
1	HeCdGe	9	1.919	HeMoGeNiCd	7	0.576
2	HeCdGe	9	1.957	HeMoGeNiCd	7	0.582
3	HeMoGeNiCd	7	1.932	HeMoGeNiCd	7	0.586
4	HeCdGe	9	1.956	HeMoGeNiCd	7	0.587
5	HeCdGe	9	1.940	HeMoGeNiCd	7	0.589
6	HeMoGeNiCd	7	1.955	HeMoGeNiCd	7	0.578
7	HeCdGe	9	1.936	HeMoGeNiCd	7	0.580
8	HeMoGeNiCd	7	1.935	HeMoGeNiCd	7	0.589
9	HeMoGeNiCd	7	1.961	HeMoGeNiCd	7	0.585
10	HeCdGe	9	1.929	HeMoGeNiCd	7	0.570
11	HeMoGeNiCd	7	9.483	HeMoGeNiCd	7	2.544
12	HeMoGeNiCd	7	9.460	HeMoGeNiCd	7	2.553
13	HeMoGeNiCd	7	9.468	HeMoGeNiCd	7	2.569
14	HeMoGeNiCd	7	9.527	HeMoGeNiCd	7	2.564
15	HeMoGeNiCd	7	9.442	HeMoGeNiCd	7	2.568
16	HeMoGeNiCd	7	9.475	HeMoGeNiCde	7	2.557
17	HeMoGeNiCd	7	9.472	HeMoGeNiCd	7	2.552
18	HeMoGeNiCd	7	9.429	HeMoGeNiCd	7	2.578
19	HeMoGeNiCd	7	9.493	HeMoGeNiCd	7	2.577
20	HeMoGeNiCd	7	9.482	HeMoGeNiCd	7	2.550

第 6 章 总结和展望

6.1 本文总结

由本次的设计，本人对于路径规划，蒙特卡洛算法和数据结构的理解更加深刻，运用也更加熟练，在以 matlab 为工具，使用基础的 matlab 语句做出了基于蒙特卡洛算法的路径规划，在基础的路径规划上，以优化为主要目标的对此功能进行改进。从各个方面减少它的时间复杂度。再从搜索范围上减少它的问题规模。从而大大提高了它的运行速率。

在测试中也可从中分析到那一部分优化对于什么样的困难有了多大的提升。从而调整它的细节来改进。而本路径规划还有许许多多的方面可以尝试。比如动态路径中的更多的目的地的比较优化。从路径预存储方面提高多次路径规划的速度。从中运用更多的数据结构的知识来加强它的效率。

6.2 未来展望

该路径规划只是在始发地的路径规划，而没有实时的功能，有可能的话，希望能够把此路径规划能够变得随着路况的实时变动，路径规划也会随之改变。

参考文献

致谢

光阴荏苒，这四年的本科学习生活让我受益匪浅，经过几个月的忙碌和工作，本次毕业设计也已经接近尾声。回想着在学校的时光，我得到了许许多多的关怀与帮助，在此向他们表达衷心的感谢。

本人在做设计期间，得到了赵梓权老师的精心指导。赵梓权老师学识渊博，他循循善诱的教导和清晰的思路给予我无尽的帮助；老师他对待我们是有问必答，尽全力帮助我们解决学习和生活上的难题，对待工作老师严谨细致，一丝不苟，是我们学习和生活的榜样；而且会积极影响我今后的学习和工作。

然后要感谢大学 4 年来的所有老师，因为有了他们的高质量的教导，为我们打下了夯实的计算机知识的基础；同时还要感谢所有的家人与朋友们，正是因为有了你们的帮助和支持。我才能顺利完成毕业设计。作为一个本科生的毕业设计，由于经验的匮乏，难免有许多考虑不周的地方，如果没有导师的督促指导。以及一起工作的同学们的支持，想要完成这个设计是难以想象的。

最后，我要感谢百忙之中帮我评审和参加答辩的各位老师，给我能够审视这 4 年来学习成果的机会。